

Manual Técnico Clean Manager

Sistema de Gestión para Tintorerías y Lavanderías

1. Arquitectura General y Tecnologías del Sistema

CleanManager está estructurado bajo una arquitectura desacoplada basada en el modelo **Cliente-Servidor**. El Frontend (Cliente) y el Backend (Servidor) funcionan de forma totalmente independiente, comunicándose exclusivamente a través de peticiones HTTP en formato JSON.

1.1. Arquitectura de Tres Capas

El sistema organiza el código y las responsabilidades en tres capas independientes:

1. **Capa de Presentación (Frontend):** Desarrollada con **React.js** y **TypeScript**. Es una SPA (*Single Page Application*) que se ejecuta directamente en el navegador del usuario. Se encarga de pintar la interfaz visual de forma dinámica y gestionar las rutas privadas de los empleados mediante *React Router DOM*.
2. **Capa de Lógica de Negocio (Backend):** Construida sobre **Node.js** con el framework **Express**. Actúa como una API RESTful encargada de recibir las peticiones del cliente, validar la seguridad, aplicar las reglas del negocio (como el cálculo del IVA) y procesar los datos.
3. **Capa de Persistencia (Base de Datos):** Gestionada con **MySQL**. Es un motor relacional que almacena toda la información de forma estructurada en tablas indexadas mediante claves primarias y foráneas, garantizando la integridad de los registros financieros y de clientes.

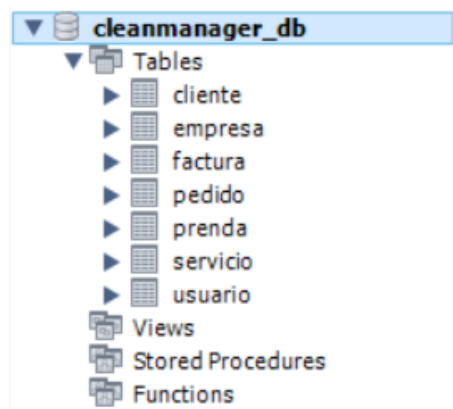
1.2. Justificación de las Tecnologías Utilizadas

- **React + TypeScript:** React permite crear componentes visuales reutilizables (como los botones de WhatsApp o las tarjetas de tarifas) y actualizar la pantalla en tiempo real sin recargar la página. TypeScript añade tipado estricto al código, lo que evita errores de desarrollo antes de subir la aplicación a producción.
- **Tailwind CSS:** Framework de estilos que permite diseñar interfaces modernas y adaptables de forma rápida mediante clases directas en las etiquetas HTML. Facilita la aplicación de relieves tridimensionales, bordes redondeados y efectos de difuminado (*backdrop blur*) sin sobrecargar las hojas de estilo.
- **Node.js + Express:** Node.js destaca por su alta velocidad y su modelo de operaciones no bloqueantes, ideal para un sistema con múltiples peticiones simultáneas desde el mostrador. Express simplifica la creación de rutas seguras para la API de forma limpia.

- **MySQL:** En un sistema de gestión con implicaciones fiscales (como el desglose de IVA y libros de facturas), las bases de datos relacionales son imprescindibles. MySQL asegura el cumplimiento de las propiedades **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad), garantizando que un cobro no se registre a medias si falla la red.
- **Chart.js:** Librería ligera seleccionada para renderizar de forma asíncrona el gráfico de donut en la pantalla de caja, proporcionando estadísticas inmediatas con un consumo mínimo de memoria del equipo local.

2. Estructura de la Base de Datos y Modelo Relacional

La base de datos de **CleanManager** (`cleanmanager_db`) está diseñada bajo el modelo relacional en **MySQL**. El pilar fundamental del diseño es la **multi-inquilinato (multi-tenant)**: todas las tiendas comparten el mismo esquema, pero la información está estrictamente aislada mediante la clave foránea `id_empresa`. Un empleado o administrador jamás podrá ver los datos de otra sucursal.



2.1. Diccionario de las 7 Tablas del Sistema

1. Tabla `empresa`

Almacena las sucursales o tintorerías registradas en la plataforma.

- `id_empresa` (INT, PK, Auto-increment): Identificador único del negocio.
- `nombre_tintoreria` (VARCHAR): Nombre comercial que se dibuja en las Navbars y tickets.

2. Tabla `usuario`

Registra las credenciales de acceso de todo el personal.

- `id_usuario` (INT, PK, Auto-increment).

- `username` (VARCHAR): Correo electrónico o identificador de login.
- `password` (VARCHAR): Contraseña (encriptada en producción).
- `rol` (ENUM): Permisos dentro del sistema ('admin' o 'empleado').
- `id_empresa` (INT, FK): Vincula al usuario con su tienda correspondiente.

3. Tabla `cliente`

Contiene el censo de clientes recurrentes del mostrador.

- `id_cliente` (INT, PK, Auto-increment).
- `nombre_completo` (VARCHAR): Nombre y apellidos del cliente.
- `telefono` (VARCHAR): Número para el disparo de alertas de WhatsApp.
- `email` (VARCHAR): Dirección para notificaciones de correo.
- `id_empresa` (INT, FK): Empresa a la que pertenece el cliente.

4. Tabla `prenda`

El catálogo maestro de ropa y tarifas base configuradas por el administrador.

- `id_prenda` (INT, PK, Auto-increment).
- `nombre_prenda` (VARCHAR): Tipo de ropa (ej. *Chaqueta de Plumas*).
- `precio_base` (DECIMAL): Tarifa base en euros sin modificaciones.
- `id_empresa` (INT, FK): Empresa que oferta esta tarifa.

5. Tabla `servicio`

Los flujos técnicos o tratamientos disponibles en el taller.

- `id_servicio` (INT, PK, Auto-increment).
- `nombre_servicio` (VARCHAR): Operación técnica (ej. *Limpieza en seco*).
- `descripcion` (TEXT): Notas opcionales del tratamiento.
- `id_empresa` (INT, FK): Empresa que realiza el servicio.

6. Tabla `pedido`

El núcleo operativo. Registra las órdenes de trabajo y su estado actual en la cadena de lavado.

- `id_pedido` (INT, PK, Auto-increment).
- `id_cliente` (INT, FK): Cliente que deja la ropa.
- `id_prenda` (INT, FK): Prenda a tratar.
- `id_servicio` (INT, FK): Tratamiento a aplicar.
- `estado_prenda` (ENUM): Estado en el taller ('SIN EMPEZAR', 'EN LAVADO', 'EN SECADO', 'LISTO PARA ENTREGA', 'ENTREGADO').
- `fecha_creacion` (TIMESTAMP): Fecha y hora de entrada de la ropa.

- `id_empresa` (INT, FK): Segmentación de seguridad de la orden.

7. Tabla `factura`

El registro legal y contable independiente para auditorías fiscales.

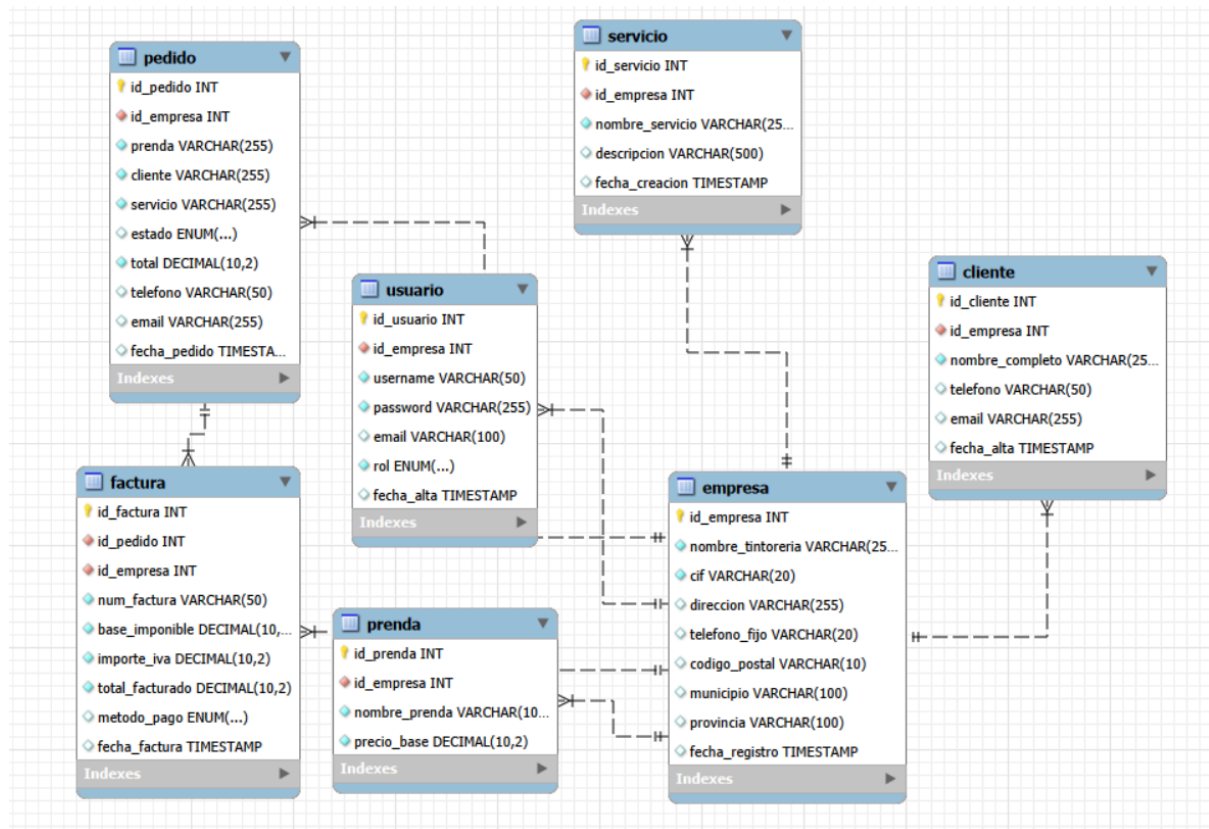
- `id_factura` (INT, PK, Auto-increment).
- `id_pedido` (INT, FK): Vínculo directo con el pedido original que se acaba de cobrar.
- `numero_factura` (VARCHAR): Código legal único indexado (ej. *FS-2026-000015*).
- `metodo_pago` (ENUM): Canal de entrada del dinero ('EFFECTIVO', 'TARJETA', 'BIZUM').
- `base_imponible` (DECIMAL): Precio neto del servicio calculado por el backend antes de impuestos.
- `iva_cuota` (DECIMAL): El 21% de impuesto devengado retenido en la transacción.
- `total_bruto` (DECIMAL): Importe final cobrado al cliente en caja.
- `fecha_emision` (TIMESTAMP): Sello de tiempo exacto del cobro.
- `id_empresa` (INT, FK): Garantiza el aislamiento contable del negocio.

2.2. Integridad Referencial y Restricciones (Constraints)

- **Relaciones en Cascada (ON DELETE CASCADE):** Si un administrador da de baja una empresa, el motor MySQL elimina automáticamente en cadena todos sus registros en las otras 6 tablas, evitando dejar datos huérfanos.
- **Consistencia del Historial Fiscal:** La tabla `factura` queda blindada de forma inalterable tras el cobro. Aunque un pedido cambie o se elimine por error en el taller, el registro contable en `factura` permanece para asegurar el cuadro de caja ante cualquier inspección.

2.3. Diagrama del Modelo Físico (Base de Datos)

A continuación, se detalla el esquema físico generado automáticamente a partir del motor MySQL. En él se puede observar la arquitectura relacional de las 7 tablas del sistema y cómo se estructuran las relaciones cardinales a través de la clave `id_empresa` para segmentar los datos de los clientes, pedidos y facturas.



2.4. Diagrama de Flujo del Ciclo de Vida del Pedido

Este diagrama modela la lógica de negocio implementada en el sistema, describiendo de forma secuencial los estados por los que transita un registro en la tabla **pedido** desde su recepción en el mostrador hasta la emisión final de la factura simplificada.



3. Control de Accesos, Rutas Protegidas y Seguridad

Este módulo del manual técnico describe los mecanismos implantados en **CleanManager** para asegurar la confidencialidad de los datos, el aislamiento entre sucursales y la restricción de funcionalidades según el nivel de privilegios del usuario.

3.1. Autenticación y Persistencia de Sesión

El control de acceso se basa en un sistema de autenticación centralizado gestionado entre el Frontend y el Backend:

1. **Petición de Login:** El usuario introduce sus credenciales en la vista de acceso. El cliente React realiza una petición `POST /api/login` enviando el identificador y la contraseña.
2. **Validación en Backend:** El servidor de Node.js consulta la tabla `usuario` en la base de datos. Si las credenciales coinciden, genera una respuesta exitosa con los datos de filiación del empleado.
3. **Persistencia Local:** El Frontend recibe el objeto de respuesta y lo almacena de forma persistente en el `localStorage` del navegador. Este objeto contiene las propiedades clave para el comportamiento de la interfaz:
 - `id_empresa`: Utilizado como filtro obligatorio en todas las consultas posteriores de datos.
 - `rol`: Define los menús y paneles a los que tiene derecho a acceder el usuario ('admin' o 'empleado').
 - `nombre_tintoreria`: Permite personalizar la cabecera dinámica de la aplicación (ej. *Tintorería Fregenal*).

3.2. Seguridad en el Frontend: Rutas Protegidas

Para evitar que un usuario manipule la URL del navegador y acceda a pantallas para las que no tiene permiso (por ejemplo, que un empleado entre al panel de configuración de tarifas del administrador), se ha implementado un sistema de **Rutas Protegidas** utilizando *React Router DOM*:

- **Componente Guard (Filtro de Ruta):** Se han creado componentes contenedores intermedios (ej. `<ProtectedRoute/>`). Este componente comprueba el `rol` guardado en el almacenamiento local antes de renderizar la vista solicitada.
- **Redirección Automatizada:**
 - Si un usuario no autenticado intenta acceder a `/dashboard`, el sistema lo redirige automáticamente a la pantalla de Login.

- Si un usuario con el rol de 'empleado' intenta forzar la URL hacia la zona de administración (ej. /configuracion-tarifas), el guardabosques de rutas bloquea la renderización y lo devuelve a su menú principal de mostrador.

3.3. Seguridad en el Backend: Aislamiento Estricto de Datos

La seguridad visual del Frontend se complementa con un blindaje en la API del Backend para asegurar una arquitectura multi-inquilinato (*multi-tenant*) infranqueable:

- **Inyección Obligatoria del `id_empresa`:** Ningún endpoint del servidor realiza operaciones generales. Todas las sentencias SQL de lectura (`SELECT`) o escritura (`INSERT`, `UPDATE`) exigen el parámetro de la empresa como condición en la cláusula `WHERE`.
 - *Ejemplo lógico:* Para cargar los servicios del mostrador, el servidor ejecuta internamente: `SELECT * FROM servicio WHERE id_empresa = ?`; Esto garantiza matemáticamente que los empleados de la *Tintorería Fregenal* jamás reciban en su JSON los registros o las tarifas configuradas por la *Tintorería Puerta Villa*, manteniendo los datos completamente estancos.

4. Endpoints Clave de la API REST

Este apartado describe el diseño de la **API REST** de **CleanManager**, detallando las rutas del servidor (endpoints) encargadas de comunicar el Frontend con la base de datos MySQL de forma estructurada. Todas las peticiones utilizan el formato JSON para el intercambio de datos.

4.1. Módulo de Autenticación y Usuarios

- **POST /api/login**
 - **Descripción:** Procesa el inicio de sesión del personal en el mostrador.
 - **Funcionamiento:** Recibe el `username` y la `password`. Ejecuta un `SELECT` en la tabla `usuario` para validar las credenciales. Si el acceso es correcto, devuelve el rol (`admin` / `empleado`) y el `id_empresa` para inicializar la sesión en el navegador.

4.2. Módulo de Configuración (Uso Exclusivo del Administrador)

- **GET /api/prendas**
 - **Descripción:** Recupera el catálogo de piezas textiles configuradas.

- **Filtro de seguridad:** Requiere el parámetro `id_empresa` en la consulta para extraer únicamente las tarifas del local activo.
- **POST /api/prendas**
 - **Descripción:** Registra un nuevo artículo con su precio base en el catálogo.
 - **Funcionamiento:** Ejecuta un `INSERT INTO prenda` inyectando el nombre de la prenda, el valor económico y el ID de la sucursal.
- **GET /api/servicios**
 - **Descripción:** Devuelve el listado de operaciones técnicas de lavado del taller.
- **POST /api/servicios**
 - **Descripción:** Registra una nueva metodología técnica de limpieza en la tabla `servicio`.
- **DELETE /api/servicios/:id**
 - **Descripción:** Elimina un tratamiento obsoleto del catálogo mediante su clave primaria.

4.3. Módulo de Operaciones e Historial (Uso de Empleados y Administrador)

- **GET /api/clientes**
 - **Descripción:** Extrae el censo completo de clientes registrados en la sucursal actual para nutrir el buscador del mostrador.
- **POST /api/clientes**
 - **Descripción:** Da de alta una nueva ficha de cliente (`INSERT INTO cliente`) recogiendo el nombre, email y teléfono para el sistema de alertas.
- **POST /api/pedidos**
 - **Descripción:** Registra la entrada de una prenda al taller desde la pantalla de nueva recepción. Almacena el registro inicial en estado `SIN EMPEZAR`.
- **PUT /api/pedidos/:id/estado**
 - **Descripción:** Modifica en caliente la fase de producción de una prenda desde el Panel de Control.
 - **Funcionamiento:** Ejecuta una sentencia `UPDATE pedido SET estado_prenda = ? WHERE id_pedido = ?`. Es el endpoint que permite transicionar los estados de taller (`EN LAVADO`, `EN SECADO`, `LISTO PARA ENTREGA`).
- **POST /api/facturas**
 - **Descripción:** Transforma un pedido en un registro contable inalterable en el momento del cobro.

- **Funcionamiento:** Calcula la base imponible y la cuota de IVA del 21% sobre el total bruto, cambia el pedido a ENTREGADO e inserta los datos financieros definitivos en la tabla factura.